Package 'iBHMM'

February 20, 2026

Title iBHMM: Bayesian Supervised Biclustering via Hidden Markov Models

Version 0.0.0.9000

Description Implements a Bayesian supervised biclustering framework using Hidden Markov Models for multi-modality data integration. The model performs joint clustering and outcome modeling via Gibbs sampling.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.2.9000

R topics documented:

Cindex 1
EvaluateTest 2
iBiclust 3
iGenDataHMM 6

Index **8**

`Cindex` *Concordance Index (C-index) for Censored Outcomes*

Description

Computes the concordance index using predicted times, observed times, and event indicators.

Usage

```
Cindex(Tpredict, Ttest, Status)
```

Arguments

`Tpredict`	Numeric vector of predicted times.
`Ttest`	Numeric vector of observed times.
`Status`	Integer/numeric vector of event indicators (1 = event observed, 0 = censored).

Value

Numeric scalar concordance index.

`EvaluateTest` *Evaluate Test-Set Predictions*

Description

Computes an outcome-specific performance metric using posterior predictive samples returned by `iBiclust`.

Usage

```
EvaluateTest(result, TestY, OutcType, threshold = 0, return_all = FALSE)
```

Arguments

`result`	A fitted model object returned by `iBiclust` containing `YpredTestSample` (posterior predictive samples for test outcomes).
`TestY`	Test-set response variable. For continuous or binary outcomes, this should be a numeric vector. For censored outcomes, this must be an $n \times 2$ matrix of the form `cbind(time, censor)`, where the second column is a censoring indicator (1 = censored, 0 = event observed).
`OutcType`	Outcome type: one of `"continuous"`, `"binary"`, or `"censored"`.
`threshold`	Numeric; classification threshold for binary outcomes applied to the posterior predictive mean. Default is 0.
`return_all`	Logical; if `TRUE`, returns a list including intermediate predictions. If `FALSE`, returns only the metric(s).

Value

For `OutcType = "continuous"`: a scalar mean square error `MSE`. For `OutcType = "binary"`: a scalar misclassification error `MCE`. For `OutcType = "censored"`: a scalar concordance index `Cindex`.

Examples

```
## Not run:
## Assume 'result' is a fitted iBiclust model with test predictions
## and 'datTest$Y' is the corresponding test response.

metric <- iEvaluateTest(
  result = result,
  TestY = datTest$Y,
  OutcType = "continuous"
)

metric

## End(Not run)
```

`iBiclust` *Bayesian Supervised Biclustering via Hidden Markov Models for Multi-Modal Data*

Description

Fits a Bayesian model-based supervised biclustering model for multi-modality datasets using an Gibb sampling algorithm. Each modality/platform is provided as a matrix with samples in rows and features in columns; internally, each matrix is optionally standardized (centered/scaled) and transposed before being passed to the native routine.

Usage

```
iBiclust(
  Method = 1,
  dataList = dataList,
  OutcType = "continuous",
  Youtcome = Youtcome,
  Cov = NULL,
  sample = 2000,
  burnin = 1000,
  K = 2,
  normalized = TRUE,
  seed = 1,
  hyperSigma = c(1, 1),
  alphaP = rep(1, K),
  delt = rep(1, 3),
  TruncVal = 0.1,
  dataListTest = NULL,
  CovTest = NULL
)
```

Arguments

`Method`	Integer specifying the modeling approach. If `Method = 1`, the model incorporates a Hidden Markov Model (HMM)
`dataList`	A list of modality matrices. Each `dataList[[t]]` is an $n \times p_t$ matrix with samples in rows and features in columns.
`OutcType`	Outcome type: one of `"continuous"`, `"binary"`, or `"censored"`.
`Youtcome`	Outcome vector/matrix. For censored outcomes, this must be an $n \times 2$ matrix `cbind(time, censor)`, where the second column represents the *censoring indicator* (1 = censored, 0 = event observed).
`Cov`	Covariate matrix of dimension $n \times p_c$.
`sample`	Number of MCMC samples to retain.
`burnin`	Number of burn-in iterations.
`K`	Number of sample clusters used in the model.
`normalized`	Logical; if `TRUE`, each modality is centered and scaled feature-wise.
`seed`	Integer random seed passed to the native routine.

`hyperSigma`	Numeric vector of length 2. The prior distribution on the state-specific variances σ^2_{dk1} and σ^2_{dk2} for $\mu_{d,j,k,l}$.
`alphaP`	Numeric vector of length `K` specifying the Dirichlet prior concentration parameters for the cluster probability vector. These parameters control the prior probability that a sample belongs to each cluster k.
`delt`	Numeric vector of length 3 giving the Dirichlet prior concentration parameters $(\delta_1, \delta_2, \delta_3)$ for each row of the transition matrix ξ.
`TruncVal`	Positive numeric truncation threshold t used in the Gibbs sampler.
`dataListTest`	Optional test set list with same structure as `dataList`.
`CovTest`	Optional test covariate matrix of dimension $n_{test} \times p_c$.

Details

Data format:

- `dataList` must be a list of length `M` (number of modalities/platforms).
- Each element `dataList[[t]]` is an $n \times p_t$ matrix: n samples (rows) and p_t features (columns).
- If `normalized = TRUE`, each modality in `dataList` is centered and scaled feature-wise using its own mean and standard deviation. If `dataListTest` is provided, the test data are standardized using the corresponding training means and standard deviations (not re-estimated from the test data).

Outcome types:

- `"continuous"`: `OutcomeType = 1`
- `"binary"`: `OutcomeType = 2`
- `"censored"`: `OutcomeType = 3`, where `Youtcome` must be an $n \times 2$ matrix (e.g., value and censoring indicator).

Test data: If `dataListTest` is provided, it must have the same list structure as `dataList`. If `normalized = TRUE`, test data are standardized using the training means/SDs.

Value

A list with posterior summaries and samples:

DIC Deviance Information Criterion returned by the sampler.

probZMean Matrix of posterior mean cluster membership probabilities ($n \times K$).

MuMean Array of posterior means of the latent mean parameters μ_{dkl} for the over- and under-expressed states ($l = 1, 2$), indexed by modality d and cluster k.

CoefySample cluster-specific outcome means with dimension $sample \times K$.

sig2yM Posterior mean of outcome variance parameter.

KappaMean List of arrays of posterior means for biclustering indicators per modality.

TransMatMean Array of posterior mean transition matrices per modality.

Betasample Matrix of sampled covariate coefficients ($sample \times p_c$).

ZtestSample Matrix of sampled test cluster labels ($sample \times n_{test}$) if test data provided.

YpredTestSample Matrix of sampled test predictions ($sample \times n_{test}$) if test data provided.

logpostsample Vector of log-posterior values of length `sample + burnin`.

Examples

```
## Not run:
## Generate synthetic multi-modality data
K <- 2
p <- c(30, 50)   # smaller for an example
n <- 50
nbrcov <- 4
OutcType <- "continuous"

dat <- iGenDataHMM(
  OrderFeatured = TRUE, K = K, NbrPlat = 2, OutcType = OutcType,
  MeanY = c(rep(1, floor(K/2)), rep(-1, K - floor(K/2))),
  pc = nbrcov, betacov = 1.0,
  MeanOverExpre = (1:K) / 2, MeanUnderExpre = -(1:K) / 2,
  p = p, seed = 1, n = n,
  SigmaOE = rep(1, K), SigmaUE = rep(1, K)
)

## Generate test data
ntest <- 20
datTest <- iGenDataHMM(
  OrderFeatured = TRUE, K = K, NbrPlat = 2, OutcType = OutcType,
  MeanY = c(rep(1, floor(K/2)), rep(-1, K - floor(K/2))),
  pc = nbrcov, betacov = 1.0,
  MeanOverExpre = (1:K) / 2, MeanUnderExpre = -(1:K) / 2,
  p = p, seed = 2, n = ntest,
  SigmaOE = rep(1, K), SigmaUE = rep(1, K)
)

## Fit model (keep MCMC small in examples)
result <- iBiclust(
  Method = 1,
  dataList = dat$X,
  Youtcome = dat$Y,
  OutcType = OutcType,
  Cov = dat$Cov,
  sample = 200,
  burnin = 100,
  K = K,
  normalized = FALSE,
  seed = 1,
  hyperSigma = c(1, 1),
  alphaP = rep(1, K),
  delt = rep(1, 3),
  TruncVal = 0.1,
  dataListTest = datTest$X,
  CovTest = datTest$Cov
)

str(result)

## End(Not run)
```

`iGenDataHMM` *Generate Synthetic Multi-Modal Data for HMM-Based Supervised Biclustering*

Description

Simulates multi-modality feature matrices, covariates, and an outcome under a cluster/state structure used by `iBiclust`. For each modality, features are assigned to one of three states within each cluster: over-expressed (`1`), under-expressed (`2`), or baseline (`3`). Outcomes can be continuous, binary, or censored.

Usage

```
iGenDataHMM(
  OrderFeatured = FALSE,
  K = K,
  NbrPlat = 2,
  OutcType = "continuous",
  MeanY = MeanY,
  pc = NULL,
  betacov = 1,
  MeanOverExpre = 1:K,
  MeanUnderExpre = -c(1:K),
  p = p,
  seed = 1,
  n = n,
  SigmaOE = rep(1, K),
  SigmaUE = rep(1, K)
)
```

Arguments

`OrderFeatured`	Logical; if `TRUE`, uses a deterministic ordering for feature membership in each modality (features 1–100 over-expressed and 101–200 under-expressed). If `FALSE`, feature membership is randomized.
`K`	Integer; number of clusters.
`NbrPlat`	Integer; number of modalities/platforms/views.
`OutcType`	Outcome type: one of `"continuous"`, `"binary"`, or `"censored"`.
`MeanY`	Numeric vector of length `K`; cluster-specific mean outcome levels.
`pc`	Integer; number of covariates to generate. Must be a positive integer.
`betacov`	Numeric; magnitude of the covariate coefficients. Coefficients are assigned as `+(betacov)` for the first half and `-(betacov)` for the remainder.
`MeanOverExpre`	Numeric vector of length `K`; means for over-expressed features in each cluster.
`MeanUnderExpre`	Numeric vector of length `K`; means for under-expressed features in each cluster.
`p`	Integer vector of length `NbrPlat`; number of features per modality.
`seed`	Integer; random seed used for simulating `X` and outcomes.
`n`	Integer; number of samples.
`SigmaOE`	Numeric vector of length `K`; standard deviations for over-expressed features.
`SigmaUE`	Numeric vector of length `K`; standard deviations for under-expressed features.

Details

Cluster assignment: Samples are assigned to `K` clusters in approximately equal-sized blocks.

Feature membership (`kappajk`): For each modality m, a $p_m \times K$ matrix is generated with entries in `{1,2,3}`. If `OrderFeatured = TRUE`, features 1–100 are set to state 1 (over-expressed) and features 101–200 to state 2 (under-expressed). Otherwise, 100 over- and 100 under-expressed features are sampled at random for each modality (requires `p[m] >= 200`).

Outcome generation: A latent mean `MeanY[Zk[i]]` is assigned to each sample and combined with a linear covariate effect. Continuous outcomes add Gaussian noise; binary outcomes threshold a noisy latent score; censored outcomes return an $n \times 2$ matrix `cbind(time, censor)`, where the second column is a censoring indicator (1 = censored, 0 = event observed).

Value

A list with components:

Y Outcome vector of length `n` for continuous/binary outcomes, or an $n \times 2$ matrix for censored outcomes `cbind(time, censor)`, where the second column is the censoring indicator (1 = censored, 0 = event observed).

X List of length `NbrPlat`; each element is an $n \times p_m$ matrix of simulated features.

kappajk List of length `NbrPlat`; each element is a $p_m \times K$ matrix with entries in `{1,2,3}` indicating over/under/baseline feature membership for each cluster.

Zk Integer vector of length `n` giving true cluster assignments in `1:K`.

Cov Numeric matrix of dimension $n \times pc$ containing generated covariates.

Examples

```
set.seed(1)
K <- 2
p <- c(300, 500)
n <- 100
dat <- iGenDataHMM(
  OrderFeatured = TRUE, K = K, NbrPlat = 2, OutcType = "continuous",
  MeanY = c(1, -1), pc = 4, betacov = 1.0,
  MeanOverExpre = (1:K) / 2, MeanUnderExpre = -(1:K) / 2,
  p = p, seed = 1, n = n,
  SigmaOE = rep(1, K), SigmaUE = rep(1, K)
)
str(dat)
```

Index

Cindex, 1

EvaluateTest, 2

iBiclust, *2*, 3, *6*
iGenDataHMM, 6